

MODULE 08

会话、JSONL、恢复、回滚与 Remote Control

append-only 事件链、UUID 图、SessionStore、sidechain 与跨设备表层

持久化的核心不是保存最终回答，而是保存足以重建对话、任务、压缩边界、文件恢复点和子 agent 关系的事件序列。

研究基线：Claude Code v2.1.236（稳定版） | Python Agent SDK v0.2.141（tag） | 历史源码观察 v2.1.88
资料核验日期：2026-08-19 | 中文深度研究版

目录

- 1. 三条持久化通道
- 2. JSONL 事件的概念 schema
- 3. Session discovery 的轻量扫描
- 4. 重建对话链
- 5. Resume、continue、branch/fork
- 6. Subagent sidechains
- 7. SessionStore：把本地 JSONL 投影到外部存储
- 8. TranscriptMirrorBatcher 的可靠性取舍
- 9. Checkpoint 与 transcript 的关系
- 10. Remote Control：远程只是表层，本地仍是执行权威
- 11. 数据生命周期与隐私

阅读提示：内部模块名以历史源码观察解释，当前功能以官方文档与 v0.2.141 SDK 源码校验。

1. 三条持久化通道

通道	格式	用途
Session transcript	每会话 append-only JSONL	对话、工具、compact、metadata
Prompt history	history.jsonl	跨会话上箭头/输入召回
Subagent sidechain	session-id/subagents/**.jsonl	隔离子 agent 历史与 parent link
Checkpoint store	文件备份/编辑历史	按 user message UUID 恢复文件
Auto memory	MEMORY.md + topic files	跨会话知识，而非逐轮历史

Session 是与项目目录绑定的已保存 conversation。CLI、Desktop、web、VS Code 的历史表层不同；CLI 本地状态按 project key 组织。[O11, S6]

2. JSONL 事件的概念 schema

示例 | 概念化字段；不同事件 subtype 字段不同

```
{
  "type": "user|assistant|system|result|...",
  "uuid": "entry-id",
  "parentUuid": "previous-logical-entry",
  "sessionId": "session-id",
  "timestamp": "...",
  "message": {"role": "...", "content": [...]},
  "parent_tool_use_id": "optional-subagent-link"
}
```

事件不是简单数组。`parentUuid` 形成逻辑链，progress/meta/compact/content-replacement 等事件会穿插。读取器必须恢复可见 conversation chain，而不是把每行直接展示。SDK `message_parser` 进一步把 wire dict 转成 User/Assistant/System/Result/Task/RateLimit 等 typed messages。[S4, S6]

3. Session discovery 的轻量扫描

`sessions.py` 使用 lite read：读取文件 head/tail、mtime、size，从有限窗口提取 session id、cwd、first prompt 和 last-wins metadata，再排序/分页。这样 `/resume` 不必解析所有巨大 transcript。历史上“首个 prompt > 16KB 导致 session 消失”的 bug 说明索引窗口是假设，不是事实来源。[S6, O1]

- session id 必须做 UUID 校验，防止目录遍历。
- project path 会规范化并编码为目录 key；worktree 需要回退匹配。
- 同一 session 可能在多个路径候选出现，需要按 mtime/可信目录去重。
- 搜索列表可以近似，真正恢复前必须完整校验 transcript。

4. 重建对话链



`_build_conversation_chain()` 从候选 leaf 回溯 parentUuid，并处理 compact boundary 的逻辑 backpointer；`_is_visible_message()` 过滤 progress/UI-only 条目；`_to_session_message()` 输出面向调用方的稳定结构。fork 时还要重映射 UUID 与 logicalParentUuid，清理源 session 特有字段。 [S6]

为什么 append-only

写入简单、崩溃时通常只损坏尾行、易审计、易镜像。代价是查询和 mutation 复杂：rename/tag 也是追加 metadata，fork 要重写链，压缩要用逻辑替换而不是就地覆盖。

5. Resume、continue、branch/fork

动作	语义	安全点
continue	恢复当前目录最近会话	无历史则新会话
resume ID	恢复指定 session	重新应用当前权限策略
fork_session	复制历史并生成新 session id	避免覆盖原链
resume_session_at	从某 entry 截断恢复	可能丢未观察事件
resume_drops_turn	声明要丢弃的 prompt turn	CLI 校验截断范围只属于该 turn

SDK 对 truncating resume 增加 guard：如果截断范围含排队用户消息或 task notification，而不只属于指定 turn，就拒绝恢复。这个设计把“我认为只回退最后一轮”升级为可验证不变量。 [S5]

6. Subagent sidechains

子 agent transcript 位于主 session 相邻目录，可递归包含 workflows/子 agent。metadata sidecar 保存 agentType、worktreePath、parent agent/tool 关系。读取时构建独立链，输出 `parent_tool_use_id` 和 `parent_agent_id`，使 UI/SDK 能把 sidechain 与父 Agent tool_use 对齐。 [S1, S6, S7]

sidechain 的价值是隔离：主会话只需摘要和进度，不承担子 agent 的完整 token 历史；需要审计时又能独立读取。

7. SessionStore：把本地 JSONL 投影到外部存储

接口	语义
append(key, entries)	按顺序追加 transcript/mirror entries
load(key)	取完整 entry list，用于恢复
list_sessions(project_key)	列出 session id 与 mtime
list_session_summaries	可选的低成本索引
list_subkeys	发现 subagent/workflow sidechains
delete	删除一个 session key

SessionStore 不是另一个 agent database，而是 transcript persistence protocol。恢复时 SDK 把 store 内容 materialize 到临时 `CLAUDE_CONFIG_DIR`，复制/重定向认证和必要 settings，再让 CLI 用原生 resume 逻辑读取。这样不必在 SDK 中重写 CLI 的完整恢复语义。 [S5, S7]

8. TranscriptMirrorBatcher 的可靠性取舍

- 默认 batched：每 turn/result 前 flush，或超过 500 entries / 1 MiB 阈值。
- eager：每 frame 触发后台 drain，但仍不阻塞 read loop。
- 同 file_path 的 entries 合并，保持 enqueue 顺序。
- 适配器瞬时错误有界重试；timeout 不重试，因为原 I/O 可能稍后仍落盘，重试会重复。
- 失败以 mirror_error system message 暴露；语义是 at-most-once，不伪装强一致。 [S5, S8]

分布式系统含义

无法取消的外部 I/O 超时后，调用方不知道写入是否成功。此时自动重试会把“不确定”变成“可能重复”。正确做法是用幂等 key/sequence number，或明确报告 uncertain outcome。

9. Checkpoint 与 transcript 的关系

Checkpoint key 通常是 user message UUID：conversation 事件标出恢复位置，文件 snapshot 存实际内容。rewind conversation 和 rewind files 可以独立执行。SDK 的 `RewindFilesResult` 还报告 skipped links，防止 symlink/hard-link/parent relocation 被静默恢复到错误目标。 [O10]

Fork 是历史复制；Rewind 是在原会话内回到恢复点；Git revert 是工程历史操作。三者不可互换。

10. Remote Control：远程只是表层，本地仍是执行权威

Remote Control 把 browser/mobile 连接到本机 Claude Code session；文件系统、MCP、tools 和项目配置仍在本机执行。断网/睡眠后会重连，状态更新可排队；Remote UI 与 terminal 可同时输入。 [O21]

对象	所在地
代码与 shell	本机
Claude Code runtime	本机

对象	所在地
远程 UI	claude.ai / mobile
消息同步/会话发现	远程控制通道
授权	仍由本机会话 permission policy 约束

误区

Remote Control 不等于 cloud execution。若本机离线、cwd 不存在或 MCP 只在本机，远端页面不能魔法般继续执行。

11. 数据生命周期与隐私

- 用 `cleanupPeriodDays` 管理 session transcript；auto memory 有不同保留规则。
- `claude project purge --dry-run` 先预览，再删除项目 transcript、task、debug、edit history 等。
- SessionStore 外部化后要自定义 retention、encryption、tenant isolation 和 deletion。
- 日志里可能含用户 prompt、源码片段、工具输出和路径；OTel/镜像前做数据分类。
- 恢复临时目录可能含凭据副本；无论成功、异常或取消都要清理。 [O20, S7]

参考资料与证据等级

[O10] 官方 | [Checkpointing](#)

<https://code.claude.com/docs/en/checkpointing>

[O11] 官方 | [Sessions](#)

<https://code.claude.com/docs/en/sessions>

[O21] 官方 | [Remote Control](#)

<https://code.claude.com/docs/en/remote-control>

[S4] 开源源码 | [Message parser](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/message_parser.py

[S6] 开源源码 | [Session reader](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/sessions.py

[S7] 开源源码 | [Session resume materialization](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/session_resume.py

[S8] 开源源码 | [Transcript mirror batcher](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/transcript_mirror_batcher.py

[S5] 开源源码 | [SDK types and options](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/types.py

方法说明

官方材料用于确认当前稳定行为；SDK 源码固定在发布 tag v0.2.141；社区材料只用于解释 v2.1.88 历史源码结构。源码行号和内部函数名可能在后续版本变化，外部契约以官方最新文档为准。